

STAT 380: Classification Technique: Tree Based Methods

UAEU

Outlook of the unit

● Prediction and Classification Approaches

- Classification Techniques
 - Logistic regression
 - Discriminant analysis
- Evaluating Performance of a Classification Technique
- **Tree-based methods: Decision trees**
 - Classification trees
 - Regression trees

Outline

1 Regression Tree

2 Classification Tree

- Various Impurity Measures
- Classification Tree Algorithm

Regression Tree

Tree-based methods: Decision trees

Prediction and classification approaches - A short overview

- Linear regression, logistic regression and multinomial logistic regression are common methods for prediction and classification.
- These methods relate a response variable to a set of explanatory variables - parametric nature.
- Linear relationships are being considered, which means that the effect on the response of a change in the explanatory variable from x to $x + 1$ is the same for any value x .
- With regression/classification trees, the models become truly nonparametric and they allow for very flexible representations.

Tree-based methods: Regression/classification trees

- Here we describe tree-based methods for regression and classification.
- These involve stratifying or segmenting the predictor space into a number of simple regions.
- Since the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of approaches are known as decision-tree methods.
- Decision trees can be applied to both regression and classification problems.

Pros and Cons: Regression/classification trees

- + Tree-based methods are simple and useful for interpretation.
- However they typically are not competitive with the best supervised learning approaches in terms of prediction accuracy.
- + Hence we also discuss bagging, random forests (and boosting). These methods grow multiple trees which are then combined to yield a single consensus prediction.
- +/- Combining a large number of trees can often result in dramatic improvements in prediction accuracy, at the expense of some loss interpretation.

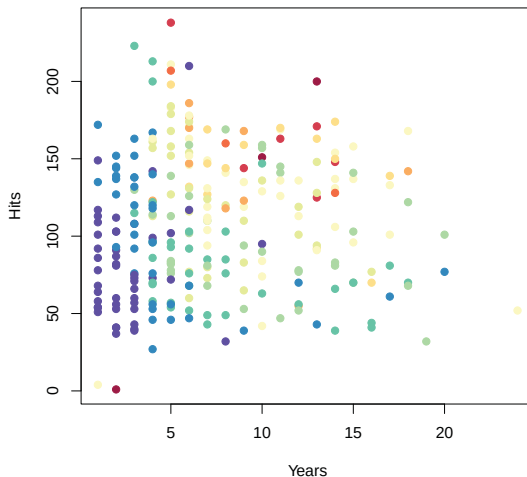
Predicting baseball players' salaries using regression trees

In order to motivate regression trees, we begin with a simple example.
Example:

- We use the *Hitters* data set (R-package: ISLR) to predict a baseball player's salary based on the variable *years* and *hits*.
- Years is the number of years that he has played in the major leagues.
- Hits is the number of hits that he made in the previous year.
- We log-transform salary so that its distribution has a more symmetric shape (salary is measured in thousands of dollars).

Predicting baseball players' salaries using regression trees

How would you stratify the data?



Salary is color-coded from low (blue, green) to high (yellow, red)

Predicting baseball players' salaries using regression trees

How would you stratify the data?

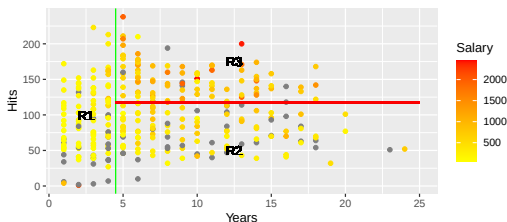


Predicting baseball players' salaries using regression trees

- A regression tree for predicting the log salary based on the number of years (played in the major leagues) and the number of hits (made in the previous year).
- At a given internal node, the label (of the form $X_j < t_k$) indicates the left-hand branch emanating from that split, and the right-hand branch corresponds to $X_j \geq t_k$.
- The split at the top of the tree results in two large branches. The left-hand branch corresponds to $Years < 4.5$, and the right-hand branch corresponds to $Years \geq 4.5$.
- The tree has two internal nodes (each decision defines a node) and three leaves/ terminal nodes. The number in each leaf is the mean of the response for the observations that fall there.

Predicting baseball players' salaries using regression trees

Overall, the tree stratifies or segments the players into three regions (terminal nodes or leaves) of predictor space:



$$R_1 = \{X | \text{Years} < 4.5\}, R_2 = \{X | \text{Years} \geq 4.5, \text{Hits} < 117.5\},$$

$$R_3 = \{X | \text{Years} \geq 4.5, \text{Hits} \geq 117.5\}$$

Terminology for trees

- In keeping with the tree analogy, the regions R_1 , R_2 , and R_3 are known as *terminal nodes*.
- Decision trees are typically drawn upside down, in the sense that the leaves are at the bottom of the tree.
- The points along the tree where the predictor space is split are referred to as *internal nodes*.
- In the hitters tree, the two internal nodes are indicated by the text $Years < 4.5$ and $Hits < 117.5$.

Interpretation of results

- *Years* is the most important factor in determining *Salary*, and players with less experience earn lower salaries than more experienced players.
- Given that a player is less experienced, the number of *Hits* that he made in the previous year seems to play little role in his *Salary*.
- Among players who have been in the major leagues for five or more years, the number of *Hits* made in the previous year does affect *Salary*, and players who made more *Hits* last year tend to have higher salaries.
- Surely an over-simplification, but compared to a regression model, it is easy to display, interpret and explain.

Regression trees

Building regression trees: Segmentation of the space

The process of building a regression tree consists of mainly two steps:

- 1 We divide the predictor space - that is, the set of possible values for $X_1, X_2, \dots, X_p$ - into J distinct and non-overlapping regions, $R_1, R_2, \dots, R_J$.
- 2 For every observation that falls into the region R_j , we make the same prediction, which is simply the mean of the response values for the training observations in R_j .

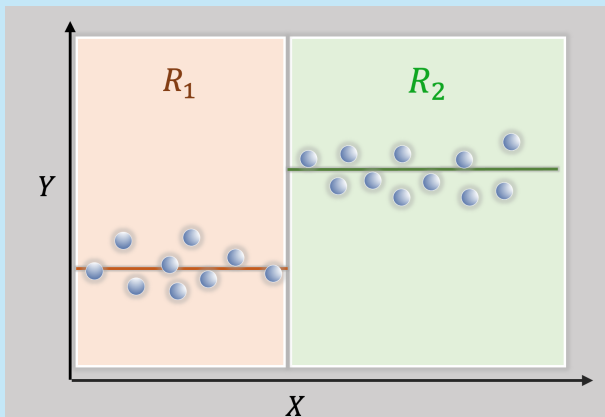
Comment to step 1): The regions could have any shape. However, we choose to divide the predictor space into high-dimensional rectangles, or boxes, for simplicity and for ease of interpretation of the resulting predictive model.

Regression Tree: Fitted Equation

Let $(y_i, \underline{\mathbf{x}}_i) \in \mathbb{R} \times \mathbb{R}^p$ for $i = 1, \dots, n$ are observed data. The mathematical representation of the fitted equation of a Regression Tree is $\hat{y} = f(\underline{\mathbf{x}})$, where

$$f(\underline{\mathbf{x}}) = \sum_{j=1}^J c_j \mathbb{I}(\underline{\mathbf{x}} \in R_j)$$

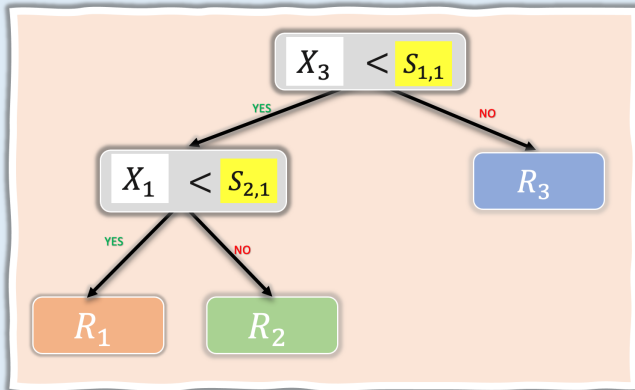
Example with Only one Covariate, Two Regions



$$f(\underline{\mathbf{x}}) = \sum_{j=1}^J c_j \mathbb{I}(\underline{\mathbf{x}} \in R_j)$$

$$\text{SSE} := \sum_{i=1}^n (y_i - f(\underline{\mathbf{x}}_i))^2 = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{c}_j)^2$$

$$\hat{c}_j = \frac{1}{n_j} \sum_{i \in R_j} y_i$$



How do we decide which variable to select while creating the binary split? i.e. which one among $\{X_1, \dots, X_p\}$

How do we select the cut off S ?

$$s \in \mathbb{R}$$

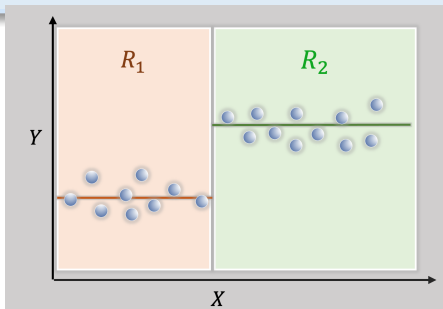
$$j = 1, \dots, p,$$

$$x_j < s$$

$$R_1(j, s)$$

$$x_j \geq s$$

$$R_2(j, s)$$



$$s \in \left\{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \dots \quad s_n = x_{j,n} \right\}$$

$$j = 1, \dots, p$$

$$x_j < s$$

$$R_1(j, s_i)$$

$$x_j \geq s$$

$$R_2(j, s_i)$$

$$C(j, s_i) := \sum_{i \in R_1(j, s_i)} \left(y_i - \hat{y}_{R_1} \right)^2 + \sum_{i \in R_2(j, s_i)} \left(y_i - \hat{y}_{R_2} \right)^2$$

$$s \in \left\{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \dots \quad s_n = x_{j,2} \right\}$$

 $j = 1, \dots, p$

$$s \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \dots \dots \dots \quad s_n = x_{j,n} \}$$

		i=1	i=2		i=n-1	i=n
$j = 1, \dots, p$	j=1					
	j=2					
	:			$C(j, s_i)$		
	j=p					

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

$$S \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \dots \quad s_n = x_{j,n} \}$$

 $j = 1, \dots, 8$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
x1	1282	2560	2770	2019	2692	2767	2185	2261
x2	2235	2744	2734	2613	2769	2770	2341	2715
x3	2284	313	1392	1580	1965	1207	2770	2463
x4	2325	2679	2341	2019	2735	2770	2560	2765
x5	2769	2700	2695	2019	2491	2284	2770	2760

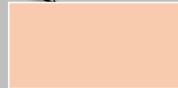
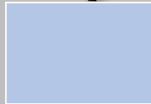
	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

S E (		$R_1 = R_{1,j}$	$R_2 = R_{2,j}$	$R_3 = R_{3,j}$	)		
f = 1...8							
x1	1282	2560	2770	2819	2682	2767	2385
x2	2235	2744	2784	2813	2769	2770	2341
x3	2284	313	1302	1580	1963	1207	2775
x4	2325	2679	2541	2819	2735	2770	2360
x5	2769	2703	2695	2819	2491	2284	2770



YES

NO



	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	49	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

$S \in \{$	$x_1 = x_{1,1}$	$x_1 = x_{1,2}$	$x_1 = x_{1,3}$	$x_1 = x_{1,4}$	$x_1 = x_{1,5}$	$x_1 = x_{1,6}$	$x_1 = x_{1,7}$	$x_1 = x_{1,8}$	$\}$
i=1	1282	2560	2770	2939	2602	2767	2385	2261	
i=2	2235	2744	2794	2893	2760	2770	2343	2715	
i=3	2284	313	1302	1580	1963	1207	2775	2483	
i=4	3325	2678	2341	2939	2735	2770	2588	2785	
i=5	2789	2700	2695	2939	2491	2284	2775	2780	

 $X_3 < 42$

YES

NO

	i=1	i=2	i=3
y	41	45	40
x1	86	93	66
x2	199	176	164
x3	27	29	24
x4	132	150	105
x5	506	540	504

 R_1

	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8
y	41	4	21	45	1	0	40	3
x1	86	60	53	93	67	59	66	76
x2	199	196	179	176	175	160	164	188
x3	27	42	48	29	53	52	24	56
x4	132	117	102	150	141	101	105	114
x5	506	537	528	540	521	505	504	523

	$S \in \{ \begin{matrix} x_1 = a_{1,1} & x_2 = a_{1,2} & \dots & x_8 = a_{1,8} \end{matrix} \}$							
$j = 1, \dots, 8$	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
x_1	1282	2590	2770	2010	2082	2787	2185	2201
x_2	2235	2744	2794	2613	2769	2770	2341	2715
x_3	2284	315	1302	1580	1905	1207	2770	2403
x_4	2325	2678	2341	2019	2735	2770	2646	2765
x_5	2768	2790	2695	2019	2491	2284	2770	2700

 $X_3 < 42$

YES

NO

	j=1	j=2	j=3
y	41	45	40
x1	86	93	66
x2	199	176	164
x3	27	29	24
x4	132	150	105
x5	506	540	504

 R_1

	i=1	i=2	i=3	i=4	i=5
y	4	21	1	0	3
x1	60	53	67	59	76
x2	196	179	175	160	188
x3	42	49	53	52	56
x4	117	102	141	101	114
x5	537	528	521	505	523

$S \in \{ \begin{matrix} A_1 = B_1 & A_2 = B_2 & \dots & A_n = B_n \end{matrix} \}$

	i=1	i=2	i=3	i=4	i=5
x1	225	299	281	10	289
x2	296	205	257	299	287
x3	290	296	281	145	285
x4	262	257	270	280	225
x5	296	149	257	299	205

 $X_1 < 59$

YES

NO

 R_2 R_3

Building regression trees: Segmentation of the space

How should we divide the predictor space?

The goal is to find boxes $R_1, R_2, \dots, R_J$ that minimize residual sum of squares

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

where $\hat{y}_{R_j}$ is the mean response for the training observations within R_j .

- Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes.
- For this reason, we take a top-down approach that is known as recursive binary splitting.
- The best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

Building regression trees: Segmentation of the space

- We first select (both) the predictor X_j and the cutpoint s such that splitting the predictor space into the regions $R_1(j, s) = \{X | X_j < s\}$ and $R_2(j, s) = \{X | X_j \geq s\}$ leads to the greatest possible reduction in RSS .
- We minimize the equation:

$$\min_{j,s} \left[\sum_{i: X_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: X_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2 \right]$$

- Next, we repeat the process, looking for the best X_j and cutpoint s to split the data further to minimize the RSS within each of the resulting regions. We split one of the two previously identified regions. We now have three regions.

Building regression trees: Segmentation of the space

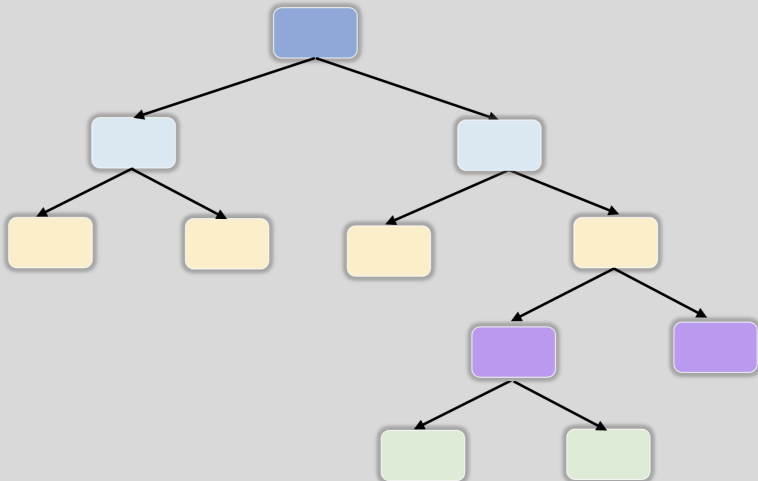
- We split one of these three regions further to minimize the RSS .
- The process continues until a stopping criterion is reached; for instance, we may continue until no region contains more than five observations.

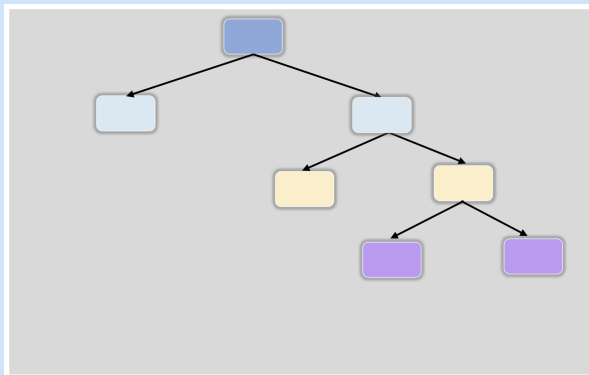
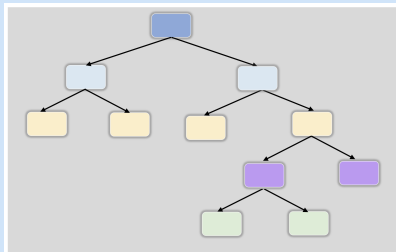
Prediction

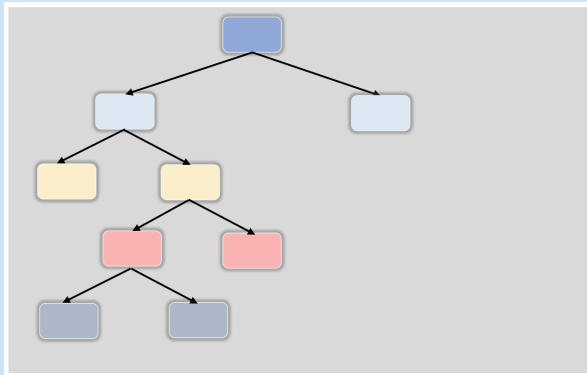
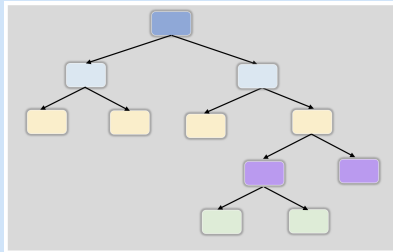
We predict the response for a given test (validation) observation using the mean of the training observations in the region to which that test observation belongs.

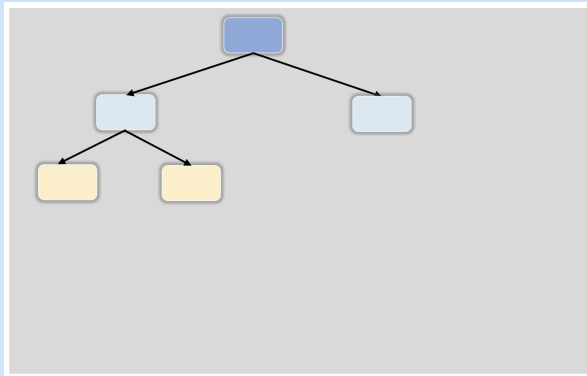
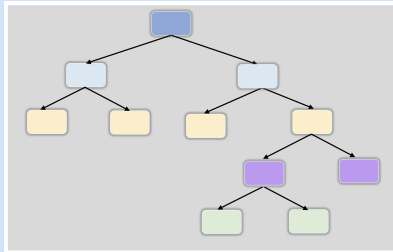
Tree, Subtree, and Tree Pruning

Subtree

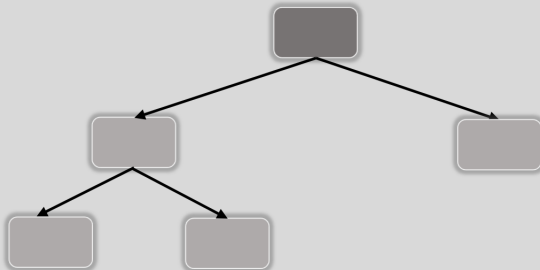
T_0 

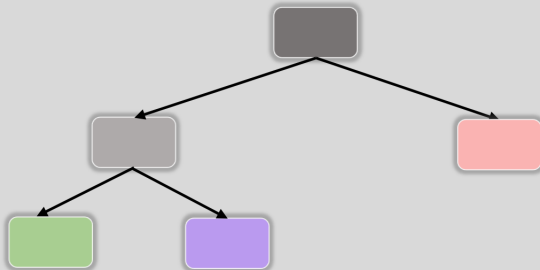


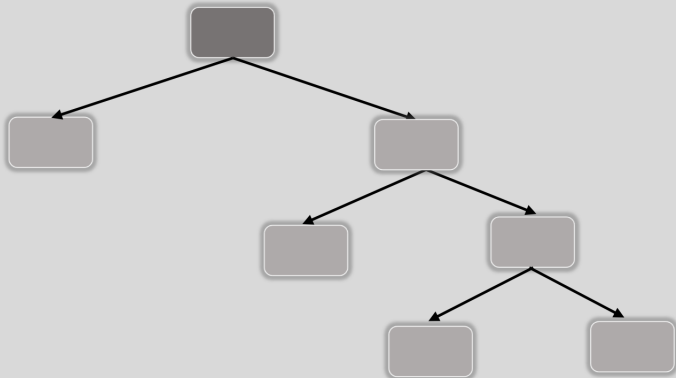


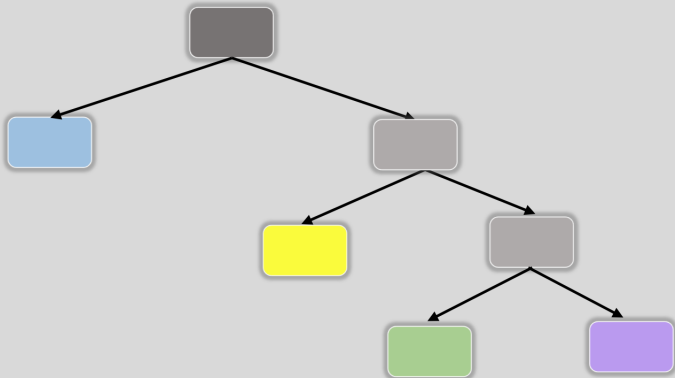


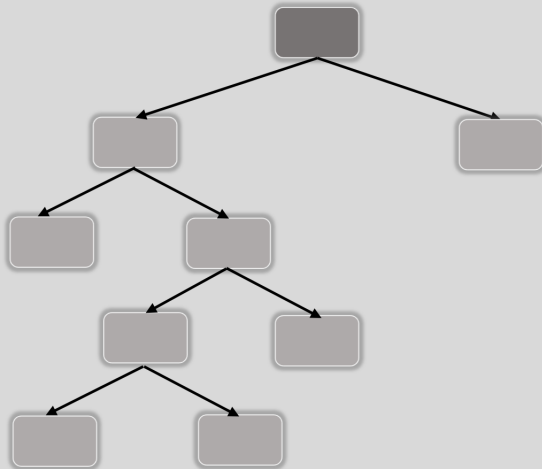
$|T|$: Order of a Tree

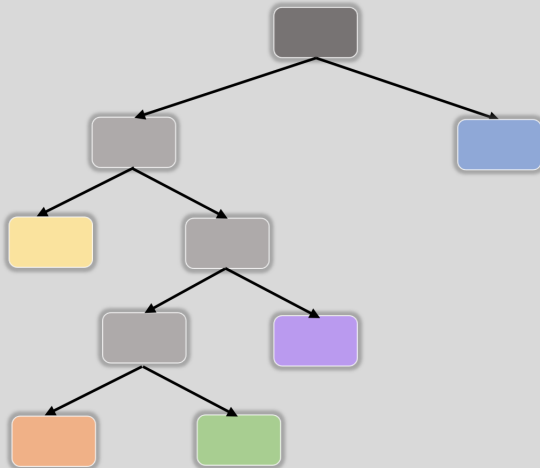


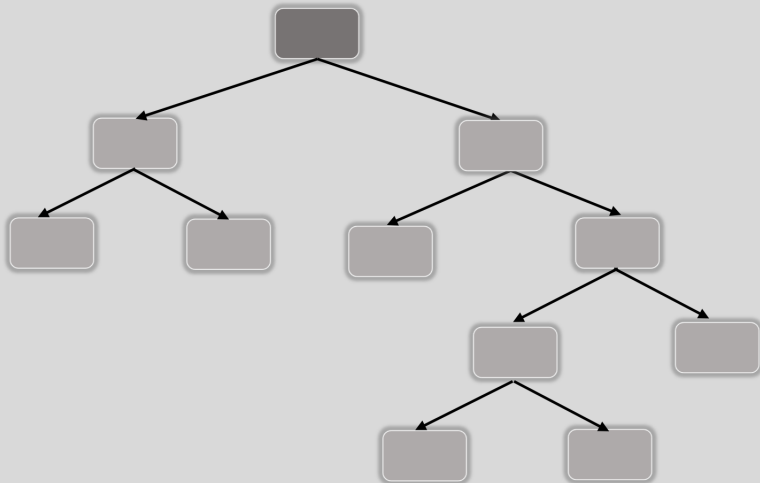


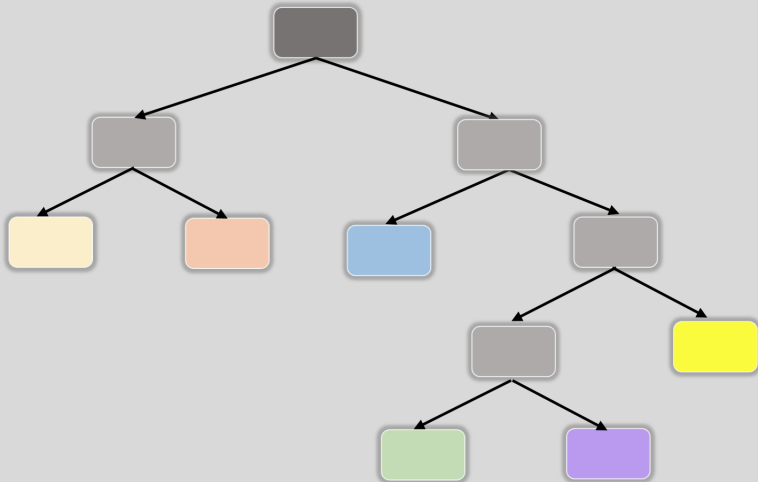












Tree Pruning

$$Cost(T) := \sum_{j=1}^{|T|} \sum_{i \in R_j} (y_i - \hat{c}_j)^2 + \alpha |T| \quad \text{where} \quad \hat{c}_j = \frac{\sum_{i \in R_j} y_i}{n_j}$$

$$T_{\alpha, \text{opt}} = \underset{T \subseteq T_0}{\text{Arg min}} Cost(T)$$

Summary: Constructing a regression tree

- ① Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
- ② Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α .
- ③ Use K -fold cross-validation to choose α . That is, divide the training observations into K folds. For each $k = 1, \dots, K$:
 - Build a regression tree (for a given α) on all $K - 1$ th folds of the training data.
 - Evaluate the mean squared prediction error on the data in the left-out k th fold, as a function of α .

Average the results for each value of α , and pick α to minimize the average error.

- ④ Return the subtree from Step 2 that corresponds to the chosen value of α .

Prediction and classification methods: Example in R

Regression trees in R: Boston housing data

The `rpart()`-command constructs a regression tree and returns the results as an object.

```
# Loading libraries and the data
```

```
library(rpart)
```

```
library(partykit)
```

```
# Run a regression tree using the rpart command
```

```
fit<-rpart(formula, data, method, minsplit, cp, subset)
```

- `minsplit` specifies min. number of units that must exist in a node in order for a split to be attempted.
- `cp` defines the complexity parameter α in the pruning of the tree.
- `subset` specifies the units in the training data set from the available data.

Regression trees in R: Boston housing data

Grow a general regression tree with

Crime rate + River 1/0 + Number of rooms + Teacher/Pupil ratio

```
# Change complexity parameter alpha to 0 - full tree
> fit<-rpart(MEDV~CRIM+CHAS+RM+PTRATIO, data=Daten,
  minsplit=10, cp=0, subset=inTrain)
# Display the results of cross-validation
> printcp(fit)
```

Variables actually used in tree construction:

```
[1] CHAS      CRIM      PTRATIO RM
```

Root node error: 34771/406 = 85.643

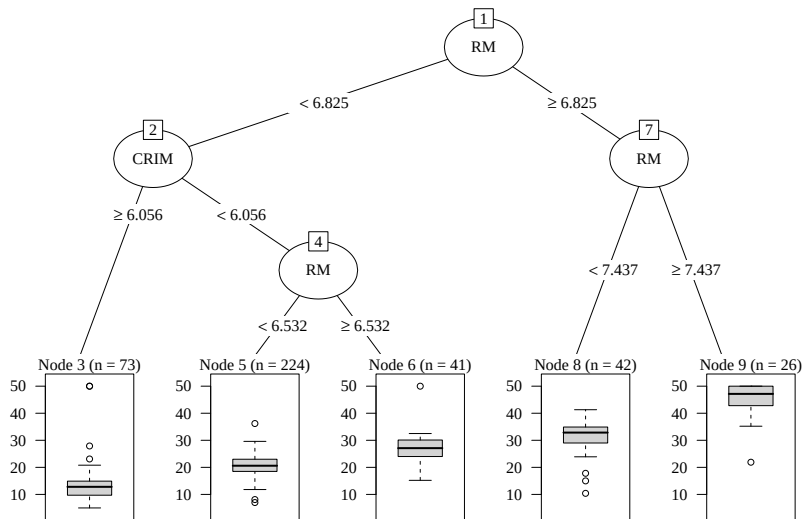
	CP	nsplit	rel error	xerror	xstd
1	0.45424798	0	1.00000	1.00491	0.092344
2	0.10154523	1	0.54575	0.59452	0.063519
3	0.09118098	2	0.44421	0.50018	0.070210
.		.			
57	0.00014848	74	0.13146	0.43197	0.075392
58	0.00013117	75	0.13131	0.43179	0.075389
59	0.00000000	76	0.13118	0.43207	0.075387

Regression trees in R: Boston housing data

The `prune()`-command prunes the regression tree. The tree can be visualized by `plot()`-command.

```
> # Visualize cross-validation results
> plotcp(fit)
>
> # Prune the tree
> fit$cptable[which.min(fit$cptable[, "xerror"]), "CP"]
[1] 0.02605373
> pfit<- prune(fit, cp=0.02605373) # from cptable
>
> # Plot the pruned regression tree
> plot(as.party(pfit))
```

Regression trees in R: Boston housing data



Regression trees in R: Boston housing data

Compare the performance of the pruned tree with the full tree on the validation data.

```
# Predictions for the validation data
> pred_v_tree<-predict(fit,newdata=vali)
> pred_v_ptree<-predict(pfit,newdata=vali)

# Evaluate performance
> accuracy(pred_v_tree,vali$MEDV)
      ME      RMSE      MAE      MPE      MAPE
Test set -0.6629  5.8775  4.001  -9.270  21.3263
> accuracy(pred_v_ptree,vali$MEDV)
      ME      RMSE      MAE      MPE      MAPE
Test set -0.6485  5.6786  3.881  -9.676  20.3017
```

Outline

1 Regression Tree

2 Classification Tree

- Various Impurity Measures
- Classification Tree Algorithm

Classification Tree

Classification trees

Classification trees - Some first comments

- A **classification tree** is very similar to a regression tree, except that it is classification used to predict a **qualitative response** rather than a quantitative one.
- We predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs.
- We are often interested not only in the **class prediction** corresponding to a particular terminal node region, but also in the **class proportions** among the training observations that fall into that region.

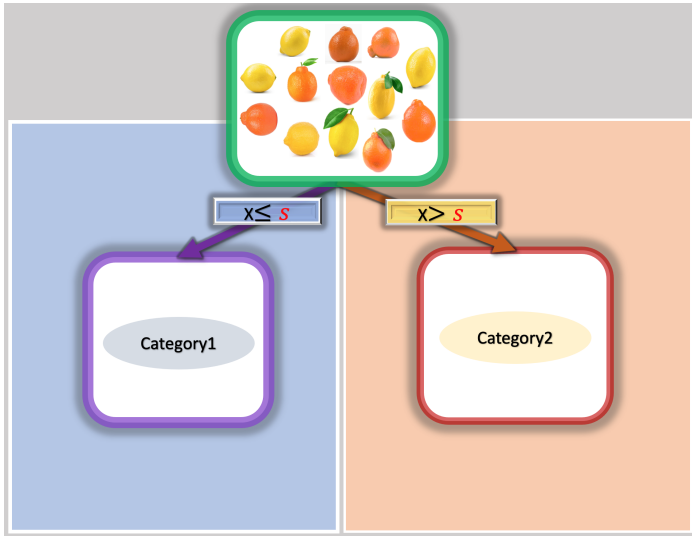
Classification Tree:Schematic Diagrams



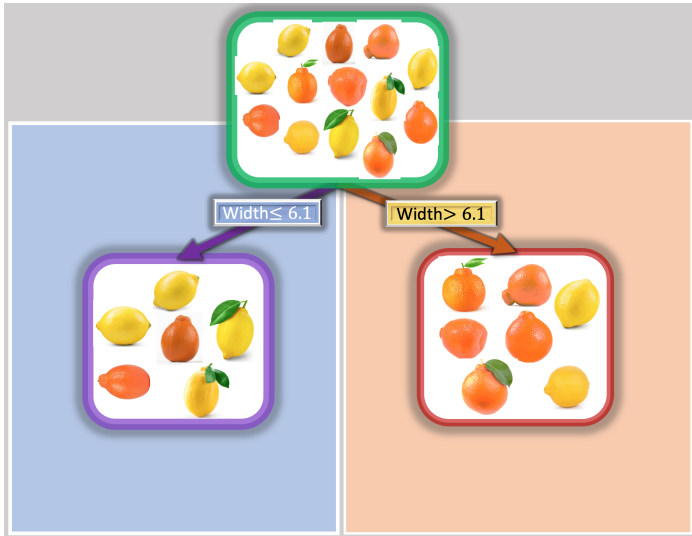
Classification Tree:Schematic Diagrams

	Width	Height			
				4.9	6.4
	6.8	7.5		7.2	7.9
	7.2	7.9		6.9	7.2
	5.9	7.8		6.8	7.0
	7.7	8.2		7.1	7.4
	6.4	8.1		6.3	7.8
	5.9	7.2		7.8	9.1

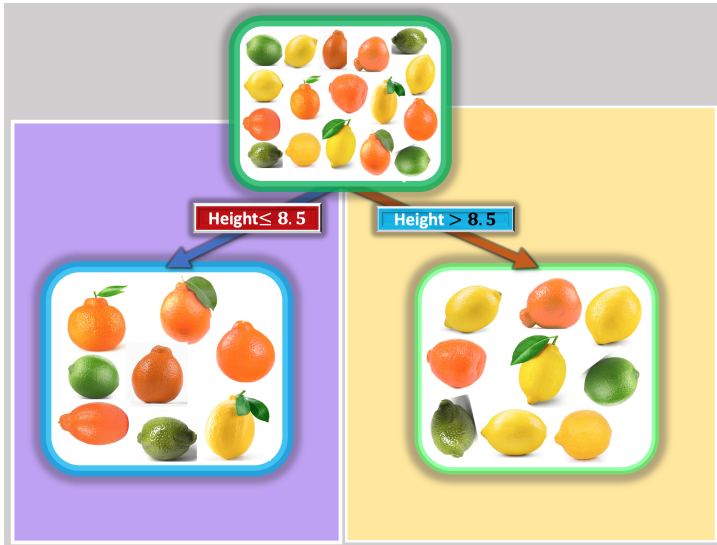
Classification Tree:Schematic Diagrams



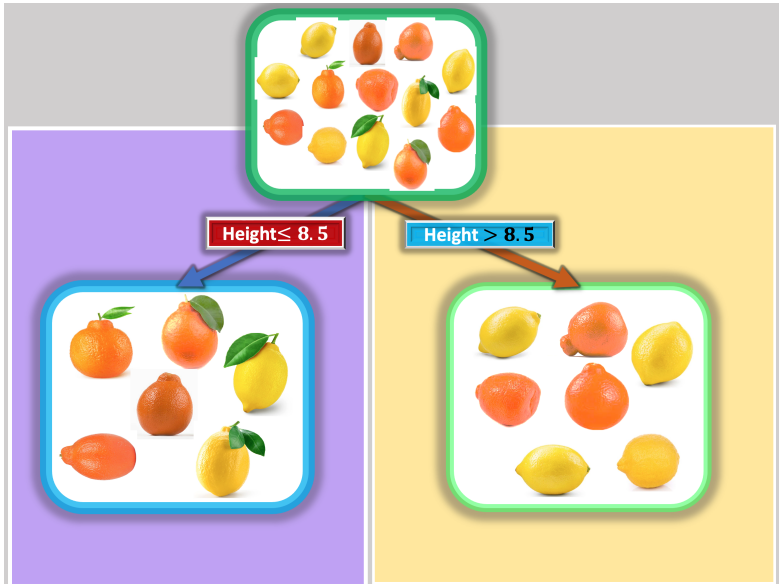
Classification Tree:Schematic Diagrams



Classification Tree:Schematic Diagrams



Classification Tree:Schematic Diagrams



Impurity Measure: Misclassification Rate

Case 1



Case 2





$$[n_1 = 4 \quad n_2 = 1 \quad n_3 = 2]$$

☐ $(n_1 = 20 \quad n_2 = 15 \quad n_3 = 5)$

☐ $\tilde{\mathbf{p}} = (.3 \quad .6 \quad .1)$

☐ $\tilde{\mathbf{p}} = \left(\frac{1}{3} \quad \frac{1}{3} \quad \frac{1}{3}\right)$

☐ $\tilde{\mathbf{p}} = (1 \quad 0 \quad 0)$

Impurity Measure: Gini Index


$$(n_1 = 20 \quad n_2 = 15 \quad n_3 = 5)$$


$$\tilde{\mathbf{p}} = (.3 \quad .6 \quad .1)$$


$$\tilde{\mathbf{p}} = \left(\frac{1}{3} \quad \frac{1}{3} \quad \frac{1}{3}\right)$$


$$\tilde{\mathbf{p}} = (1 \quad 0 \quad 0)$$

Impurity Measure: Entropy Measure

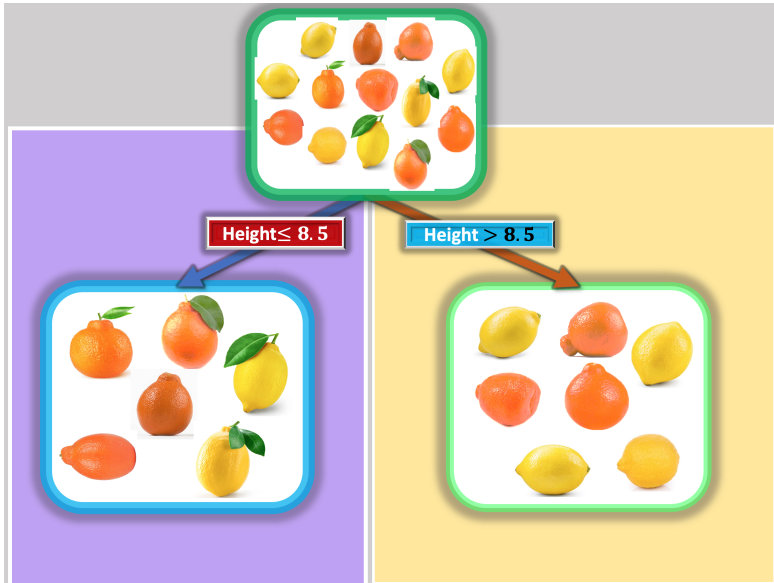
☐ $(n_1 = 20 \quad n_2 = 15 \quad n_3 = 5)$

☐ $\underline{\mathbf{p}} = (.3 \quad .6 \quad .1)$

☐ $\underline{\mathbf{p}} = \left(\frac{1}{3} \quad \frac{1}{3} \quad \frac{1}{3}\right)$

☐ $\underline{\mathbf{p}} = (1 \quad 0 \quad 0)$

Classification Tree: Schematic Diagrams



□ If we define model prediction based on the most commonly occurring class of training observations in a given region, the **classification error rate** is the proportion of the training observations that do not belong to the most common class :

$$\text{classification error rate} = 1 - \max_k \hat{p}_{m,k}$$

□ If we define model prediction based on the most commonly occurring class of training observations in a given region, the **Gini index** measures the impurity using the following formula

$$G = \sum_{k=1}^K \hat{p}_{m,k} (1 - \hat{p}_{m,k})$$

Gini index is a measure of node purity - a small value indicates that a node contains predominantly observations from a single class.

□ If we define model prediction based on the most commonly occurring class of training observations in a given region, the **cross-entropy** measures the impurity using the following formula

$$D = - \sum_{k=1}^K \hat{p}_{m,k} \log (\hat{p}_{m,k})$$

Building classification trees

- We use recursive binary **splitting** to grow a classification tree.
- In the classification setting, RSS cannot be used as a criterion for making the binary splits.
- A natural alternative to RSS is the classification error rate:

$$E = 1 - \max_k(\hat{p}_{mk}),$$

where $\hat{p}_{mk}$ represents the proportion of training observations in the m -th region that are from the k -th class.

- The **classification error rate** is simply the fraction of the training observations in that region that do not belong to the most common class.

Building classification trees - Two other measures

- The classification error is not sufficiently sensitive for tree-growing.
- The Gini index is defined by:

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

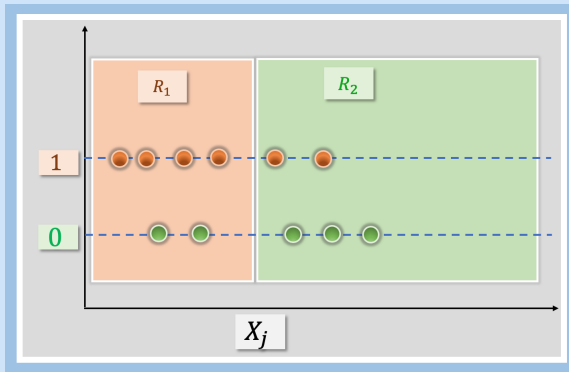
and measures the variation across the K classes (Gini is small when all $\hat{p}_{mk}$ are close to zero or one).

- Gini index is a measure of *node purity* - a small value indicates that a node contains predominantly observations from a single class.
- An alternative to the Gini index is cross-entropy

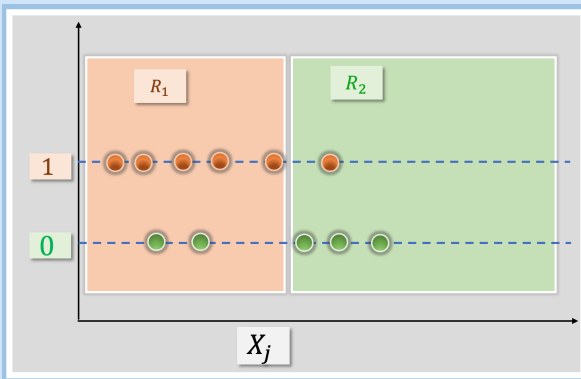
$$D = - \sum_{k=1}^K \underbrace{\hat{p}_{mk} \log(\hat{p}_{mk})}_{\leq 0}.$$

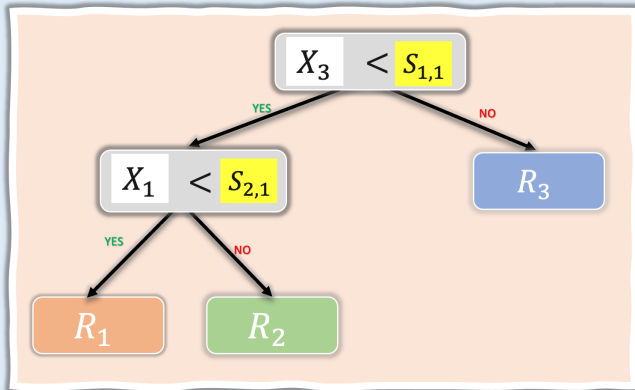
- The cross-entropy takes a value near zero if the $\hat{p}_{mk}$ are all near zero or near one (small values indicate that nodes are pure).

Classification Tree:Schematic Diagrams



Classification Tree:Schematic Diagrams





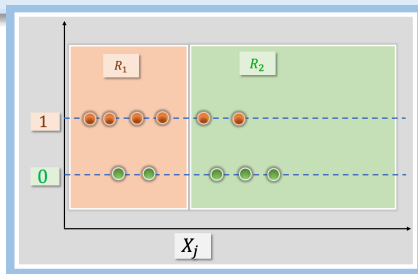
- How do we decide which variable to select while creating the binary split? i.e. which one among $\{X_1, \dots, X_p\}$
- How do we select the cut off S ?

$j = 1, \dots, p,$ $s \in \mathbb{R}$

$$x_j < s$$

 $R_1(j, s)$

$$x_j \geq s$$

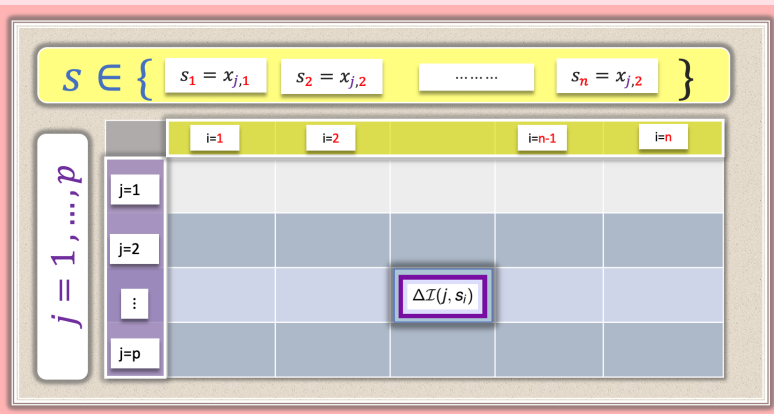
 $R_2(j, s)$ 

Let the dataset $\mathcal{D}$ is partitioned into two parts $R_1 := \{X_j \leq s\}$, and $R_2 := \{X_j > s\}$

- 1 $\mathcal{D} := R_1 \cup R_2$ denotes the entire dataset. (training data set)
- 2 $|\mathcal{D}| = n$, sample size.
- 3 $|R_1| = n_1$, number of samples in R_1 .
- 4 $|R_2| = n_2$, number of samples in R_2 .
- 5 $n_1 + n_2 = n$
- 6 $\mathcal{I}(\mathcal{D})$, $\mathcal{I}(R_1)$, $\mathcal{I}(R_2)$ denotes an *impurity measure* for $\mathcal{D}$, R_1 , and R_2 , respectively.

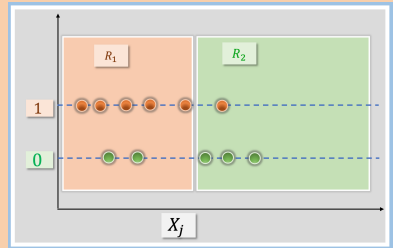
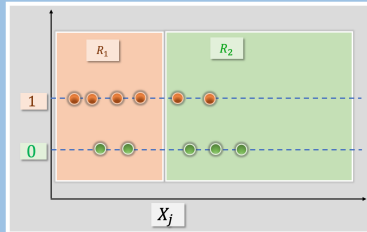
The impurity reduction/ information gain is defined to be.

$$\Delta \mathcal{I}(j, s_j) := \mathcal{I}(\mathcal{D}) - \left\{ \frac{n_1}{n_1 + n_2} \mathcal{I}(R_1) + \frac{n_2}{n_1 + n_2} \mathcal{I}(R_2) \right\}$$



$$(j_{\text{opt}}, i_{\text{opt}}) := \underset{j \in \{1, \dots, p\}, i \in \{1, \dots, n\}}{\text{Arg max}} \quad \Delta I(j, s_i)$$

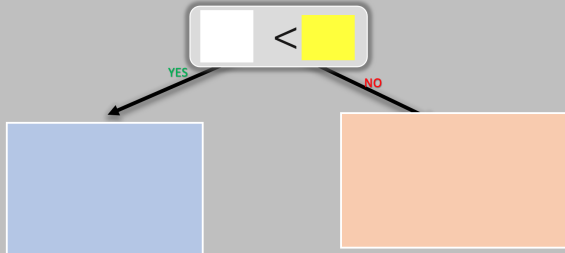
Classification Tree:Schematic Diagrams



	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10	i=11	i=12
y	1	1	0	1	0	1	0	1	1	1	0	1
x1	96	63	72	82	84	60	68	62	61	93	67	62
x2	155	189	197	182	195	176	185	172	167	173	189	162
x3	40	26	14	52	49	33	20	15	45	25	38	10
x4	108	102	128	149	108	114	104	141	145	112	108	138

$S \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \dots \quad s_n = x_{j,2} \}$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10	i=11	i=12
$j = 1, \dots, p$												
x1	0	0.08	0	0	0	0	0	0	0	0	0	0
x2	0	0.17	0	0.25	0.08	0.17	0.2	0	0	0.08	0.17	0
x3	0	0	0	0	0	0	0	0	0	0	0	0
x4	0.08	0	0	0	0.08	0	0	0	0	0	0.08	0



	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10	i=11	i=12
y	1	1	0	1	0	1	0	1	1	1	0	1
x1	96	63	72	82	84	60	68	62	61	93	67	62
x2	155	189	197	182	195	176	185	172	167	173	189	162
x3	40	26	14	52	49	33	20	15	45	25	38	10
x4	108	102	128	149	108	114	104	141	145	112	108	138

$S \in \{ s_1 = x_{j,1} \quad s_2 = x_{j,2} \quad \dots \quad s_n = x_{j,n} \}$

	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10	i=11	i=12
x1	0	0.08	0	0	0	0	0	0	0	0	0	0
x2	0	0.17	0	0.25	0.08	0.17	0.2	0	0	0.08	0.17	0
x3	0	0	0	0	0	0	0	0	0	0	0	0
x4	0.08	0	0	0	0.08	0	0	0	0	0	0.08	0

$j = 1, \dots, p$

$$X_2 < 182$$

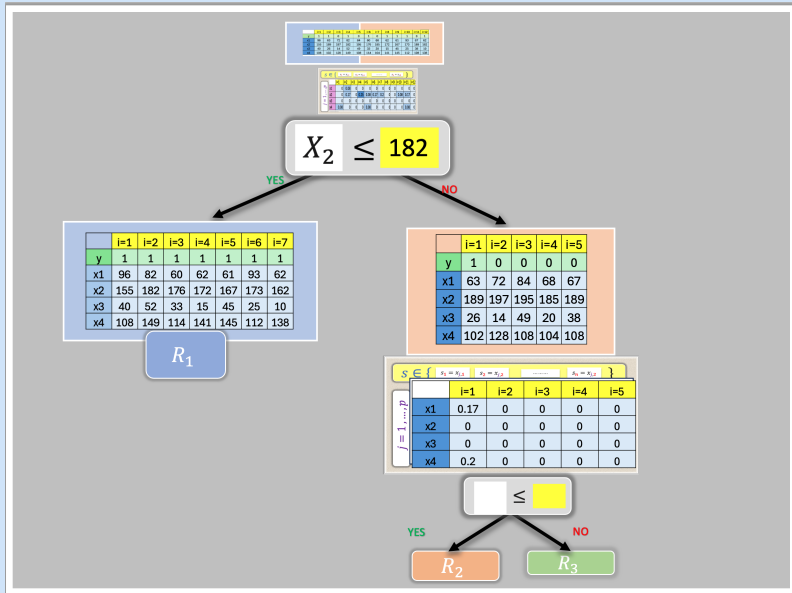
YES

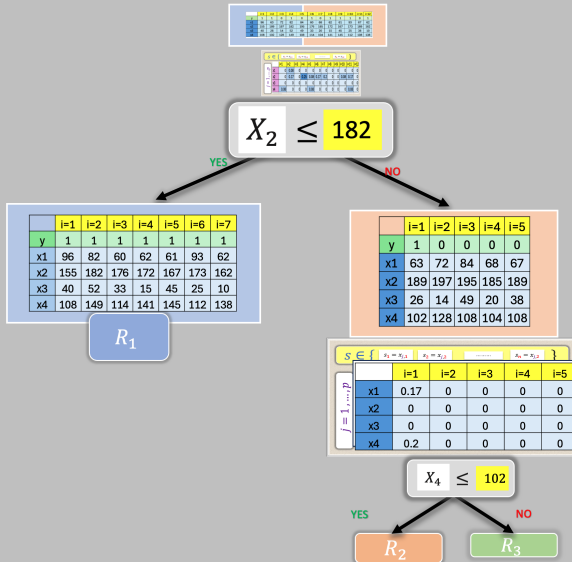
NO

	i=1	i=2	i=3	i=4	i=5	i=6	i=7
y	1	1	1	1	1	1	1
x1	96	82	60	62	61	93	62
x2	155	182	176	172	167	173	162
x3	40	52	33	15	45	25	10
x4	108	149	114	141	145	112	138

 R_1

	i=1	i=2	i=3	i=4	i=5
y	1	0	0	0	0
x1	63	72	84	68	67
x2	189	197	195	185	189
x3	26	14	49	20	38
x4	102	128	108	104	108





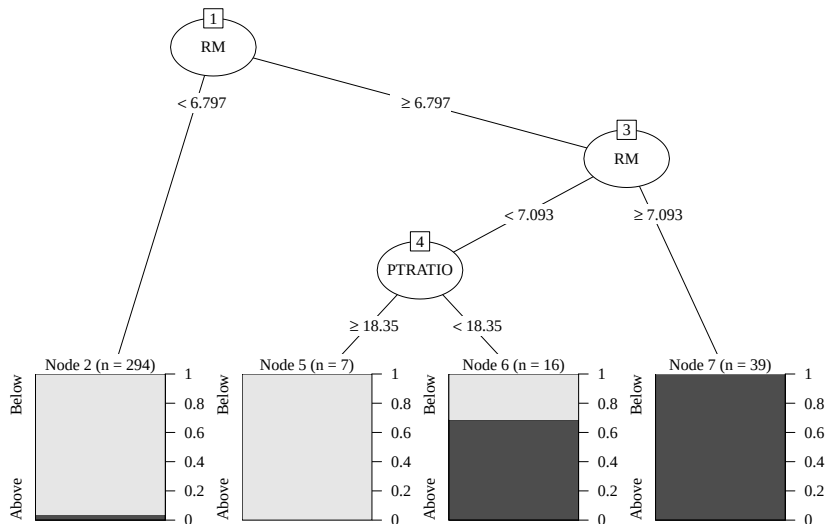
When to Stop

If tree growing is never stopped, it can result in perfectly pure nodes of one observation each. This is likely to overfit the training data and give poor prediction on the test set.

Strategy to overcome too complex models:

- stop if decrease in impurity is smaller than a threshold
- grow a very large tree and prune it back ('cut branches')

Classification trees in R: Boston housing data



Classification trees in R: Boston housing data

Compare the performance of the pruned tree with the full tree on the validation data.

```
# Predictions for the validation data
> pred_v_tree<-predict(fit,newdata=vali,type="class")
> pred_v_ptree<-predict(pfit,newdata=vali,type="class")

# Evaluate performance
> confusionMatrix(pred_v_tree, vali$MEDV_Fac)
Prediction Below Above
      Below      120      6
      Above       8     16
              Accuracy : 0.9067

> confusionMatrix(pred_v_ptree, vali$MEDV_Fac)
Prediction Below Above
      Below      122      3
      Above       6     19
              Accuracy : 0.94
```

Advantages and disadvantages of trees

- + Trees are very easy to explain to people.
- + Some people believe that decision trees more closely mirror human decision-making than do the regression and classification approaches.
- + Trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small).
- Trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches.
- The decision trees suffer from high variance.

However, by aggregating many decision trees, the predictive performance of trees can be substantially improved.

Thank You